

2ª Avaliação Banco de Dados Relacional – Gabriel Kodato e Gustavo Rosa

Stored Procedures:

1) Problema:

Controlar onde cada usuário da empresa está trabalhando em um determinado dia (Remoto ou Presencial)

A procedure deve:

Trazer uma lista de registros ligando usuário_local com usuário (Inner join), mostrando nome, setor, role e o local (Remoto/Presencial) na data registrada

-Código SQL:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_lista_usuario_local(  
    IN p_usuario_id INT,  
    IN p_data DATE  
)  
  
BEGIN  
    SELECT  
        u.id AS usuarioId,  
        u.nome,  
        u.setor,  
        u.role,  
        ul.id AS usuarioLocalId,  
        ul.local,  
        ul.data  
    FROM usuario_local ul  
    INNER JOIN usuario u ON u.id = ul.usuario_id  
    WHERE (p_usuario_id IS NULL OR ul.usuario_id = p_usuario_id)  
        AND (p_data IS NULL OR ul.data = p_data)  
    ORDER BY ul.data DESC, u.nome;  
  
END //
```

```
DELIMITER ;
```

- Retorno Obtido:

```
8  
9 • -- todos os usuários em todas as datas  
0 CALL sp_lista_usuario_local(NULL, NULL);  
1
```

usuarioId	nome	setor	role	usuarioLocalId	local	data
2	Hildy Gumley	Logística Reversa	operacional	21	Remoto	2026-11-08
35	Orran Gueny	Transporte Rodoviário	operacional	42	Presencial	2026-11-01
33	Alyda Pochon	Logística Reversa	operacional	41	Remoto	2026-09-10
13	Rossie Lovegrove	Logística Reversa	usuario	6	Presencial	2026-09-03
36	Tad Caesman	Transporte Rodoviário	comercial	24	Presencial	2026-08-30
7	Chane Dantesia	Rastreamento de Cargas	operacional	47	Presencial	2026-08-23
7	Chane Dantesia	Rastreamento de Cargas	operacional	50	Presencial	2026-08-16
30	Jonell Greenman	Armazenagem	admin	29	Presencial	2026-08-09
44	Udell Bew	Gestão de Frota	admin	13	Remoto	2026-06-01
11	Zondra Vaughten	Transporte Rodoviário	comercial	32	Remoto	2026-05-28
14	Deva Chapier	Rastreamento de Cargas	comercial	34	Remoto	2026-05-20
49	Donall Bartolomvis	Transporte Aéreo	comercial	7	Remoto	2026-05-14
43	Anne-corinne Gir...	Logística Reversa	comercial	19	Presencial	2026-04-23
18	Mil Cleal	Logística Reversa	usuario	10	Remoto	2026-04-07
29	Kilian Grunbaum	Gestão de Frota	admin	8	Remoto	2026-03-12
47	Colby Geach	Rastreamento de Cargas	operacional	27	Remoto	2026-02-14
9	Duocid Willsoos	Gestão de Frota	comercial	43	Presencial	2026-02-03

2) Problema:

Acompanhar todas as cotações realizadas por cada cliente, mostrando o nome, CNPJ e o status, valor e validade de cada cotação

-Código SQL:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_lista_cotacoes_cliente(  
    IN p_cliente_id INT  
)  
  
BEGIN  
    SELECT  
        c.Id          AS clienteId,  
        c.NomeFantasia,  
        c.CNPJ,  
        co.id         AS cotacaoId,  
        co.status,  
        co.valor_total,  
        co.data_criacao,  
        co.data_validade  
    FROM cotacao co  
    INNER JOIN cliente c ON co.id_cliente = c.Id  
    WHERE (p_cliente_id IS NULL OR c.Id = p_cliente_id)  
    ORDER BY c.NomeFantasia, co.data_criacao DESC;  
  
END //
```

```
DELIMITER ;
```

-Retorno:

```
5 • CALL sp_lista_cotacoes_cliente(NULL);
```

clientId	NomeFantasia	CNPJ	cotacaoId	status	valor_total	data_criacao	data_validade
46	Abatz	55667788000145	46	Aprovada	9733.23	2025-12-06 00:00:00	2026-05-26
26	Babbleset	00456789000123	26	Aprovada	2485.49	2025-12-17 00:00:00	2025-07-03
1	Blogtag	27894567000190	1	Em revisão	8092.06	2025-06-26 00:00:00	2026-12-09
14	Bluejam	01928374000156	14	Recusada	1463.83	2025-12-28 00:00:00	2025-05-03
23	Bluezoom	27894567000190	23	Recusada	1587.27	2025-11-08 00:00:00	2025-11-04
47	Brainverse	00456789000123	47	Pendente	4173.06	2025-02-22 00:00:00	2025-09-28
15	Browsebug	55667788000145	15	Recusada	6632.34	2026-08-19 00:00:00	2026-02-14
31	Devcast	11222333000181	31	Recusada	5556.63	2025-08-05 00:00:00	2026-01-31
36	Devpulse	01928374000156	36	Expirada	6232.64	2025-06-01 00:00:00	2025-03-16
13	Divavu	01928374000156	13	Expirada	8132.54	2025-03-23 00:00:00	2026-02-27
50	Dynabox	55667788000145	50	Pendente	8464.30	2026-08-08 00:00:00	2025-03-04
49	Edgeblab	27894567000190	49	Aprovada	9008.01	2026-06-21 00:00:00	2025-06-04
3	Einti	00456789000123	3	Recusada	3417.38	2026-02-10 00:00:00	2025-08-24
19	Fivedub	00456789000123	19	Aprovada	8368.40	2026-08-06 00:00:00	2026-05-21
34	Fivespan	00456789000123	34	Expirada	4864.14	2026-11-04 00:00:00	2026-06-17
5	Flashspan	01928374000156	5	Aprovada	9974.52	2026-09-07 00:00:00	2025-10-12
16	Flobuo	11222333000181	16	Aprovada	4166.03	2026-05-25 00:00:00	2026-08-29

3)Problema:

Facilitar o acompanhamento dos convidados de cada evento, listando os usuários, status (Pendente, Confirmado, Recusado) e o motivo (quando houver)

-Código SQL:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_lista_convidados_evento(  
    IN p_evento_id INT  
)  
  
BEGIN  
    SELECT  
        e.id      AS eventoId,  
        e.titulo  AS eventoTitulo,  
        e.dataHora AS eventoData,  
        u.id      AS usuarioId,  
        u.nome    AS usuarioNome,  
        ec.status AS conviteStatus,  
        ec.motivo AS motivoResposta,  
        ec.criadoEm AS conviteCriadoEm  
    FROM evento_convidado ec  
    INNER JOIN evento e    ON ec.evento_id = e.id  
    INNER JOIN usuario u   ON ec.usuario_id = u.id  
    WHERE (p_evento_id IS NULL OR e.id = p_evento_id)  
    ORDER BY e.dataHora DESC, ec.status, u.nome;  
END //
```

```
DELIMITER ;
```

-Retorno:

1 • CALL sp_lista_convidados_evento(NULL);

2

eventoId	eventoTitulo	eventoData	usuarioId	usuarioNome	conviteStatus	motivoResposta	conviteCriadoEm
2	Dia da Logística	2020-09-16	28	Harriet Shear	Pendente	NULL	2025-09-08 00:00:00
2	Dia da Logística	2020-09-16	10	Nicolai Bosnell	Recusado	Outros compromissos	2025-11-19 00:00:00
5	Logística em Foco	2020-07-27	10	Nicolai Bosnell	Recusado	Outros compromissos	2026-06-17 00:00:00
8	Dia da Logística	2019-10-19	38	Charlean Andreix	Recusado	Não sinto-me confortável	2025-03-23 00:00:00
35	Encontro Logístico	2019-09-27	2	Hildy Gumley	Pendente	NULL	2026-10-07 00:00:00
35	Encontro Logístico	2019-09-27	31	Dallis Collaton	Confirmado	NULL	2026-08-10 00:00:00
12	Logística em Foco	2018-10-20	38	Charlean Andreix	Recusado	Não concordo com as condições	2026-12-18 00:00:00
25	Conferência de Colaboradores	2018-07-05	35	Orran Gueny	Confirmado	NULL	2026-06-01 00:00:00
42	Logística em Foco	2017-11-19	50	Coraline MacGiolla Pheadair	Pendente	NULL	2026-05-08 00:00:00
42	Logística em Foco	2017-11-19	2	Hildy Gumley	Confirmado	NULL	2025-08-24 00:00:00
42	Logística em Foco	2017-11-19	43	Anne-corinne Girardin	Recusado	Outros compromissos	2026-02-12 00:00:00
42	Logística em Foco	2017-11-19	30	Jonell Greenman	Recusado	Não vejo benefícios	2026-01-27 00:00:00
9	Encontro Logístico	2017-11-07	37	Danni Battaille	Recusado	Não concordo com as condições	2025-12-10 00:00:00
34	Dia da Logística	2016-06-30	30	Jonell Greenman	Recusado	Não sinto-me confortável	2025-01-30 00:00:00
47	Encontro Logístico	2016-01-25	24	Freddy Mahomet	Recusado	Outros compromissos	2026-09-28 00:00:00
40	Encontro Logístico	2015-06-07	7	Chane Dantesia	Recusado	Outros compromissos	2026-08-10 00:00:00
6	Conferência de Colaboradores	2014-12-24	23	Tailor Paoelsen	Pendente	NULL	2025-04-27 00:00:00

4) Problema:

Permitir consultar o histórico de categorias de um cliente específico, mostrando quando o cliente mudou de categoria, qual categoria era e a observação associada.

O uso de LEFT JOIN permite registros com categoria ou id de cliente NULL, mantendo histórico mesmo em caso de alteração nas categorias ou exclusão de cliente.

-Código SQL:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_historico_categorias_cliente(  
    IN p_cliente_id INT  
)  
BEGIN  
    SELECT  
        c.Id AS clienteId,  
        c.NomeFantasia,  
        cc.categoria,  
        rc.dataRegistro,  
        rc.observacao  
    FROM registro_cliente rc  
    LEFT JOIN cliente_categoria cc ON rc.categoriaId = cc.id  
    LEFT JOIN cliente c ON rc.clienteId = c.Id  
    WHERE (p_cliente_id IS NULL OR c.Id = p_cliente_id)  
    ORDER BY rc.dataRegistro DESC;  
END //
```

```
DELIMITER ;
```

-Retorno:

Result Grid		 Filter Rows:			Export: 	Wrap Cell Content: 
	clienteId	NomeFantasia	categoria	dataRegistro	observacao	
▶	8	Riffwire	Potencial	2026-12-26	Novo contato registrado	
	12	Jaxspan	Em Negociacao	2026-12-24	E-mail enviado ao cliente	
	50	Dynabox	Prospect	2026-12-16	Contato telefônico realizado	
	6	Roodel	Prospect	2026-12-12	Novo contato registrado	
	25	Myworks	Em Negociacao	2026-12-04	Documento recebido	
	21	Yakijo	Potencial	2026-11-04	Visita realizada	
	24	Oloo	Inicial	2026-10-14	Contato telefônico realizado	
	31	Devcast	Prospect	2026-09-27	E-mail enviado ao cliente	
	50	Dynabox	Manutencao	2026-08-28	Visita realizada	
	12	Jaxspan	Potencial	2026-08-06	Novo contato registrado	
	1	Blogtag	Potencial	2026-07-12	Contato telefônico realizado	
	17	Meemm	Potencial	2026-06-12	Novo contato registrado	
	5	Flashspan	Follow Up	2026-06-01	E-mail enviado ao cliente	
	25	Myworks	Inicial	2026-05-21	Contato telefônico realizado	
	47	Brainverse	Inicial	2026-05-19	Aguardando retorno do clie...	
	33	Vidoo	Em Negociacao	2026-03-18	Documento recebido	
	14	Blueiam	Manutencao	2026-03-13	Contato telefônico realizado	

5) Problema:

Visualizar e consultar todos os agendamentos de cada cliente.

-Código SQL:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_agendamentos_por_cliente(  
    IN p_cliente_id INT  
)  
  
BEGIN  
    SELECT  
        c.Id AS clienteId,  
        c.NomeFantasia AS clienteNome,  
        c.CNPJ,  
        a.id AS agendamentoId,  
        a.titulo AS agendamentoTitulo,  
        a.dataAgendamento,  
        a.localizacao,  
        a.descricao  
    FROM agendamento_cliente a  
    INNER JOIN cliente c ON a.clienteId = c.Id  
    WHERE (p_cliente_id IS NULL OR c.Id = p_cliente_id)  
    ORDER BY a.dataAgendamento DESC, c.NomeFantasia;  
END //
```

```
DELIMITER ;
```

-Retorno:

	clienteId	clienteNome	CNPJ	agendamentoId	agendamentoTitulo	dataAgendamento	localizacao	descricao
▶	13	Divavu	01928374000156	7	Reuniao com cliente	2030-12-11 00:00:00	Sala de Reunião 1	Revisão dos resultados financeiros do último se...
	1	Blogtag	27894567000190	46	Reuniao com cliente	2030-11-26 00:00:00	Videoconferência	Discussão sobre novos projetos
	37	Gabtype	01928374000156	34	Reuniao com cliente	2030-10-04 00:00:00	Sala de Reunião 2	Revisão dos resultados financeiros do último se...
	14	Bluejam	01928374000156	20	Reuniao com cliente	2030-09-16 00:00:00	Videoconferência	Brainstorming de ideias para campanha de mark...
	25	Myworks	00456789000123	48	Reuniao com cliente	2030-08-15 00:00:00	Sala de Reunião 1	Análise de desempenho da equipe
	22	Kwinu	00456789000123	23	Reuniao com cliente	2030-06-25 00:00:00	Auditório	Revisão dos resultados financeiros do último se...
	30	Twimm	11222333000181	14	Reuniao com cliente	2030-06-04 00:00:00	Cliente - Onsite	Revisão dos resultados financeiros do último se...
	36	Devpulse	01928374000156	45	Reuniao com cliente	2030-04-15 00:00:00	Sala de Reunião 2	Análise de desempenho da equipe
	29	Yambee	01928374000156	22	Reuniao com cliente	2030-03-10 00:00:00	Cliente - Onsite	Discussão sobre novos projetos
	7	Tambee	27894567000190	39	Reuniao com cliente	2029-10-21 00:00:00	Sala de Reunião 2	Análise de desempenho da equipe
	45	Plambee	27894567000190	15	Reuniao com cliente	2029-10-02 00:00:00	Café Local	Planejamento estratégico para o próximo trimes...
	46	Abatz	55667788000145	37	Reuniao com cliente	2029-04-22 00:00:00	Escritório Central	Planejamento estratégico para o próximo trimes...
	20	Topdrive	11222333000181	33	Reuniao com cliente	2029-03-02 00:00:00	Auditório	Brainstorming de ideias para campanha de mark...
	40	Jaxspan	00456789000123	32	Reuniao com cliente	2029-02-20 00:00:00	Auditório	Brainstorming de ideias para campanha de mark...
	10	Zooveo	55667788000145	30	Reuniao com cliente	2028-10-18 00:00:00	Auditório	Planejamento estratégico para o próximo trimes...
	48	Yoveo	55667788000145	6	Reuniao com cliente	2028-03-12 00:00:00	Sala de Reunião 2	Planejamento estratégico para o próximo trimes...
	40	Jaxspan	00456789000123	50	Reuniao com cliente	2028-03-11 00:00:00	Cliente - Onsite	Revisão dos resultados financeiros do último se...

6) Problema:

Permitir que o organizador de um evento rapidamente consulte todas as respostas dos participantes, incluindo a avaliação (nota), objetivo, comentários e nome do usuário.

-Código SQL:

```
DELIMITER //
```

```
> CREATE PROCEDURE sp_respostas_evento(  
    IN p_evento_id INT  
- )  
> BEGIN  
    SELECT  
        e.id AS eventoId,  
        e.titulo AS eventoTitulo,  
        e.dataHora AS eventoData,  
        u.id AS usuarioId,  
        u.nome AS usuarioNome,  
        er.avaliacao,  
        er.objetivo,  
        er.comentarios  
    FROM evento_resposta er  
    INNER JOIN evento e ON er.evento_id = e.id  
    INNER JOIN usuario u ON er.usuario_id = u.id  
    WHERE (p_evento_id IS NULL OR e.id = p_evento_id)  
    ORDER BY er.dataEvento DESC, u.nome;  
- END //
```

```
DELIMITER ;
```

-Retorno:

	eventoId	eventoTitulo	eventoData	usuarioId	usuarioNome	avaliacao	objetivo	comentarios
▶	1	Conferência de Colaboradores	2007-12-31	1	Armstrong Copin	4	Capacitação	Recomendo fortemente!
	48	Conferência de Colaboradores	2020-06-13	48	Alejandrina McAless	5	Lançamento de Produto	Excelente evento!
	46	Dia da Logística	2015-12-03	46	Alix Swalwel	1	Atualização	Equipe muito atenciosa.
	30	Encontro Logístico	2012-05-16	30	Jonell Greenman	1	Atualização	Excelente evento!
	50	Encontro Logístico	2006-09-12	50	Coraline MacGiolla Pheadair	3	Networking	Palestras muito informativas.
	33	Rota de Sucesso	2007-04-04	33	Alyda Pochon	3	Atualização	Excelente evento!
	44	Encontro Logístico	2008-09-08	44	Udell Bew	1	Networking	Ótima organização.
	26	Rota de Sucesso	2007-05-29	26	Leonhard Dunthorn	4	Capacitação	Excelente evento!
	6	Conferência de Colaboradores	2014-12-24	6	Netta Carwithim	3	Capacitação	Excelente evento!
	15	Rota de Sucesso	2009-12-31	15	Owen Greig	1	Lançamento de Produto	Excelente evento!
	36	Rota de Sucesso	2007-01-05	36	Tad Caesman	4	Networking	Excelente evento!
	40	Encontro Logístico	2015-06-07	40	Rees Petrusch	3	Capacitação	Ótima organização.
	4	Conferência de Colaboradores	2014-12-29	4	Avie Reiling	1	Capacitação	Palestras muito informativas.
	49	Dia da Logística	2013-01-12	49	Donall Bartolomivis	2	Planejamento Estratégico	Recomendo fortemente!
	31	Logística em Foco	2011-07-31	31	Dallis Collaton	2	Planejamento Estratégico	Palestras muito informativas.
	14	Rota de Sucesso	2012-04-17	14	Devva Chapier	3	Capacitação	Ótima organização.
	47	Encontro Logístico	2016-01-25	47	Colby Geach	3	Atualização	Recomendo fortemente!
	18	Conferência de Colaboradores	2005-12-26	18	Mil Cleal	3	Atualização	Equipe muito atenciosa.
	38	Conferência de Colaboradores	2020-03-13	38	Charlean Andreix	4	Planejamento Estratégico	Equipe muito atenciosa.
	45	Conferência de Colaboradores	2003-01-26	45	Ann-marie Larham	5	Planejamento Estratégico	Excelente evento!

TRIGGERS

1) Problema:

Após criar um usuário, cria um valor para ele na tabela usuário local. Isso simula a regra de negócio do sistema.

-Código SQL:

```
DELIMITER $$

CREATE TRIGGER trigger_inserir_usuario_local
AFTER INSERT ON usuario
FOR EACH ROW
BEGIN
    INSERT INTO usuario_local (usuario_id, local, data)
    VALUES (NEW.id, 'Presencial', CURDATE());
END //

DELIMITER ;
```

-Retorno:

Result Grid				
	id	usuario_id	local	data
▶	51	51	Presencial	2025-11-26
*	NULL	NULL	NULL	NULL

2) Após atualizar evento_convocado, insere valor em evento_resposta, com motivo caso a resposta seja RECUSADO

-Código SQL:

```

DELIMITER $$

CREATE TRIGGER trigger_update_evento_convocado
AFTER UPDATE ON evento_convocado
FOR EACH ROW
> BEGIN
    -- Se status foi alterado para 'Confirmado', cria uma resposta padrão em evento_resposta
> IF NEW.status = 'Confirmado' AND OLD.status <> 'Confirmado' THEN
>     INSERT INTO evento_resposta (
        evento_id,
        usuario_id,
        tituloEvento,
        dataEvento,
        objetivo,
        avaliacao,
        comentarios
    - )
    SELECT
        e.id,
        NEW.usuario_id,
        e.titulo,
        e.dataHora,
        'Participar do evento',
        FLOOR(RAND() * 5) + 1,
        'Resposta automática criada pelo sistema'
    FROM evento e
    WHERE e.id = NEW.evento_id;
- END IF;

-- Se status foi alterado para 'Recusado', define um motivo padrão
> IF NEW.status = 'Recusado' AND OLD.status <> 'Recusado' AND (NEW.motivo IS NULL OR NEW.motivo = '') THEN
    UPDATE evento_convocado
    SET motivo = 'Motivo não informado'
    WHERE id = NEW.id;
- END IF;
- END //

DELIMITER ;

```

-Retorno:

```

79 • UPDATE evento_convocado
80 SET status = 'Confirmado'
81 WHERE id = 1;
82
83 • SELECT * FROM evento_convocado WHERE id = 1;
84 • SELECT * FROM evento_resposta WHERE usuario_id = 1 AND evento_id = 1;
85
86

```

Result Grid						
Filter Rows:						
	id	usuario_id	evento_id	status	motivo	criadoEm
▶	1	26	3	Confirmado	NULL	2026-12-22 00:00:00
*	NULL	NULL	NULL	NULL	NULL	NULL

-Código SQL:

```
DELIMITER $$

CREATE TRIGGER trigger_insert_evento_e_convocado
AFTER INSERT ON evento
FOR EACH ROW
BEGIN
    INSERT INTO evento_convocado (
        usuario_id,
        evento_id,
        status,
        criadoEm
    ) VALUES (
        1,
        NEW.id,
        'Pendente',
        NOW()
    );
END $$

DELIMITER ;
```

-Retorno:

```
115 • SELECT * FROM evento_convocado
116 WHERE evento_id = (
117     SELECT id FROM evento WHERE titulo = 'Evento Trigger'
118 );
119
120
121
```



The screenshot shows a database client interface with a toolbar at the top containing icons for 'Result Grid', 'Filter Rows', 'Edit', and 'Export/Import'. Below the toolbar is a table with the following data:

id	usuario_id	evento_id	status	motivo	criadoEm
52	1	51	Pendente	NULL	2025-11-26 21:44:53

4) Problema:

-Após atualizar registro_cliente para categoria "Follow Up", este trigger cria uma cotação com o cliente envolvido

-Código SQL:

```
DELIMITER $$

CREATE TRIGGER trigger_update_registro_cliente
AFTER UPDATE ON registro_cliente
FOR EACH ROW
BEGIN
    IF NEW.categoriaId = 6 AND OLD.categoriaId <> 6 THEN

        INSERT INTO cotacao (
            id_cliente,
            data_criacao,
            data_validade,
            status,
            valor_total
        ) VALUES (
            NEW.clienteId,
            NOW(),
            DATE_ADD(CURDATE(), INTERVAL 30 DAY),
            'Pendente',
            0.00
        );

    END IF;
END $$
```

```
DELIMITER ;
```

-Retorno:

```
--
48 • -- Supõe que já exista um cliente com Id = 1
49 INSERT INTO registro_cliente (categoriaId, clienteId, dataRegistro, observacao)
50 VALUES (1, 1, CURDATE(), 'Cliente entrou como categoria inicial');
51
52 • UPDATE registro_cliente
53 SET categoriaId = 6,
54     dataRegistro = CURDATE(),
55     observacao = 'Cliente passou para categoria 6'
56 WHERE id = 1; -- use o id real do registro_cliente
57
58 -- Ver o registro_cliente atualizado
59 • SELECT * FROM registro_cliente
60 WHERE id = 1;
61
62 -- Ver a cotação criada automaticamente para esse cliente
63 • SELECT * FROM cotacao
64 WHERE id_cliente = 1
65 ORDER BY id DESC
```

Result Grid									
Filter Rows:									
Edit:    Export/Import:   Wrap Cell Content: 									
id	id_cliente	data_criacao	data_validade	status	valor_total	detalhes_frete	motivo_recusa		caminho_arquivo
1	1	2025-06-26 00:00:00	2026-12-09	Em revisão	8092.06	Morbi vestibulum, velit id pretium iaculis, diam eros...	Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.		Eu.png
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL		NULL

Após a criação de uma cotação, marca uma reunião automática para daqui 30 dias em

agendamento_cliente

-Código SQL:

```
DELIMITER $$

CREATE TRIGGER trigger_insert_cotacao_e_reuniao
AFTER INSERT ON cotacao
FOR EACH ROW
BEGIN
    INSERT INTO agendamento_cliente (
        clienteId,
        titulo,
        dataAgendamento,
        descricao,
        localizacao
    ) VALUES (
        NEW.id_cliente,
        'Reunião de Acompanhamento da Cotação',
        DATE_ADD(NEW.data_criacao, INTERVAL 3 DAY),
        'Reunião automática agendada após a criação da cotação.',
        'A confirmar'
    );
END $$

DELIMITER ;
```

-Retorno:

```
221 • SELECT * FROM agendamento_cliente
222 WHERE clienteId = 1
223 ORDER BY id DESC
```

Result Grid						
Filter Rows:						
Edit:   						
Export/Import:  						
Wrap Cell Content: 						
id	clienteId	titulo	dataAgendamento	descricao	localizacao	
46	1	Reuniao com cliente	2030-11-26 00:00:00	Discussão sobre novos projetos	Videoconferência	
*	NULL	NULL	NULL	NULL	NULL	

6) Problema:

Este trigger valida uma cotação antes de ser criada, não permitindo a criação caso o valor total seja negativo e/ou a data de validade seja inválida.

-Código SQL:

```

DELIMITER $$

CREATE TRIGGER trigger_valida_valor_cotacao
BEFORE INSERT ON cotacao
FOR EACH ROW
BEGIN
    IF NEW.valor_total < 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Valor total da cotação não pode ser negativo.';
    END IF;

    IF NEW.data_validade < DATE(NEW.data_criacao) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Data de validade não pode ser menor que a data de criação.';
    END IF;
END $$

DELIMITER $$

```

-Retorno:

```

INSERT INTO cotacao (
    id_cliente,
    data_criacao,
    data_validade,
    status,
    valor_total
) VALUES (
    1,
    NOW(),
    DATE_ADD(CURDATE(), INTERVAL 30 DAY),
    'Pendente',
    -100.00      -- valor inválido
);

```

Error Code: 1644. Valor total da cotação não pode ser negativo.

```

INSERT INTO cotacao (
    id_cliente,
    data_criacao,
    data_validade,
    status,
    valor_total
) VALUES (
    1,
    '2025-12-10 10:00:00',
    '2025-12-09', -- menor que a data_criacao
    'Pendente',
    500.00
);

```

Error Code: 1644. Data de validade não pode ser menor que a data de criação.

```

INSERT INTO cotacao (
    id_cliente,
    data_criacao,
    data_validade,
    status,
    valor_total
) VALUES (
    1,
    NOW(),
    DATE_ADD(CURDATE(), INTERVAL 30 DAY),
    'Pendente',
    1000.00
);

```

Retorno (sucesso):

#	id	id_cliente	data_criacao	data_validade	status	valor_total	detalhes_frete	motivo_recusa	caminho_arquivo_p	obs
1	51	1	2025-11-27 03:49:08	2025-12-27	Pendente	1000.00	NULL	NULL	NULL	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

FUNCTIONS

1) Função que retorna os dias que faltam para expirar a cotação

-Código SQL:

```
DELIMITER $$

CREATE FUNCTION func_dias_para_expirar(idCotacao INT)
RETURNS INT
DETERMINISTIC
> BEGIN
    DECLARE diasRestantes INT;

    -- DATEDIFF: Return the number of days between two date values
    SELECT DATEDIFF(data_validade, CURDATE())
    INTO diasRestantes
    FROM cotacao
    WHERE id = idCotacao;

    RETURN diasRestantes;
- END $$

DELIMITER ;
```

-Retorno:

```
9 • CREATE VIEW view_cotacoes_com_dias_para_expirar AS
0 SELECT
1     c.id,
2     c.id_cliente,
3     c.data_validade,
4     func_dias_para_expirar(c.id) AS dias_para_expirar
5 FROM cotacao c;
6
7 • SELECT * FROM view_cotacoes_com_dias_para_expirar;
8
9
```

ult Grid   Filter Rows: Export:  Wrap Cell			
id	id_cliente	data_validade	dias_para_expirar
11	11	2026-02-26	92
12	12	2026-01-01	36
13	13	2026-02-27	93
14	14	2025-05-03	-207
15	15	2026-02-14	80

2) Função que retorna o total de cotações existentes para um cliente específico.

-Código SQL:

```

CREATE FUNCTION func_total_cotacoes_cliente(clienteId INT)
RETURNS INT
DETERMINISTIC
> BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM cotacao
    WHERE id_cliente = clienteId;

    RETURN total;
~ END $$

DELIMITER ;

```

-Retorno:

```

1 • CREATE VIEW view_total_cotacoes_cliente_id AS
2   SELECT
3     c.Id AS clienteId,
4     c.NomeFantasia,
5     func_total_cotacoes_cliente(c.Id) AS total_cotacoes
6   FROM cliente c;
7
8 • SELECT * FROM view_total_cotacoes_cliente_id;

```

ult Grid   Filter Rows: Export:  Wrap Cell Content: 		
clienteId	NomeFantasia	total_cotacoes
1	Blogtag	5
2	Pixoboo	1
3	Einti	1
4	Linklinks	1
5	Flashspan	1

3) Função que calcula o valor médio das cotações de um cliente

-Código SQL:

```
DELIMITER $$
```

```
CREATE FUNCTION func_valor_medio_cotacoes(clienteId INT)
```

```
RETURNS DECIMAL(10,2)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE media DECIMAL(10,2);
```

```
    SELECT AVG(valor_total)
```

```
    INTO media
```

```
    FROM cotacao
```

```
    WHERE id_cliente = clienteId;
```

```
    RETURN media;
```

```
END $$
```

```
DELIMITER ;
```

-Retorno:

- ```
CREATE VIEW view_valor_medio_cotacoes AS
SELECT
 c.Id AS clienteId,
 c.NomeFantasia,
 func_valor_medio_cotacoes(c.Id) AS media_valor
FROM cliente c;
```
- ```
SELECT * FROM view_valor_medio_cotacoes;
```

clienteId	NomeFantasia	media_valor
1	Blogtag	2418.41
2	Pixoboo	4366.01
3	Einti	3417.38
4	Linklinks	3797.04
5	Flashspan	9974.52

4) Função que retorna o total de clientes em uma categoria.

-Código SQL:

DELIMITER \$\$

```
CREATE FUNCTION func_total_clientes_por_categoria(catId INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM registro_cliente
    WHERE categoriaId = catId;

    RETURN total;
END $$
```

DELIMITER ;

-Retorno:

- **CREATE VIEW** view_cliente_por_categoria **AS**
SELECT
 cc.id,
 cc.categoria,
 func_total_clientes_por_categoria(cc.id) **AS** total_clientes
FROM cliente_categoria cc;
- **SELECT** * **FROM** view_cliente_por_categoria;

Grid





Filter Rows:

Export:



Wrap Cell Content:



categoria	total_clientes
Prospect	11
Inicial	6
Potencial	10
Manutencao	10
Em Negociacao	6

5) Problema:

Função que retorna os eventos pendentes de um usuário, ou seja, os eventos que foi convidado mas ainda não respondeu.

-Código SQL:

```
DELIMITER $$
```

```
CREATE FUNCTION func_eventos_pendentes(p_usuarioId INT)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE total INT;
```

```
    SELECT COUNT(*)
```

```
    INTO total
```

```
    FROM evento_convitado
```

```
    WHERE usuario_id = p_usuarioId
```

```
        AND status = 'Pendente';
```

```
    RETURN total;
```

```
END $$
```

-Retorno:

```
6 DELIMITER ;
```

```
7 • CREATE OR REPLACE VIEW view_eventos_pendentes_por_usuario AS
```

```
8 SELECT
```

```
9     u.id AS usuario_id,
```

```
0     func_eventos_pendentes(u.id) AS total_eventos_pendentes
```

```
1 FROM usuario u
```

```
2 ORDER BY total_eventos_pendentes DESC;
```

```
3
```

```
4 • SELECT * FROM view_eventos_pendentes_por_usuario;
```

```
5
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
usuario_id	total_eventos_pendentes			
1	2			
24	2			
37	1			
33	1			
42	1			

6) Função que retorna o gênero de um usuário, como solicitado pela empresa NeweLog para dados administrativos.

-Código SQL:

```
DELIMITER $$

CREATE FUNCTION func_genero_usuario(p_usuarioId INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE generoRetornado VARCHAR(20);

    SELECT genero
    INTO generoRetornado
    FROM usuario
    WHERE id = p_usuarioId;

    RETURN generoRetornado;
END $$

DELIMITER ;
```

-Retorno:

- **CREATE OR REPLACE VIEW** view_genero_usuarios **AS**
SELECT
 u.id **AS** usuario_id,
 func_genero_usuario(u.id) **AS** genero
FROM usuario u;
- **SELECT * FROM** view_genero_usuarios;

Grid		Filter Rows:		Export:		Wrap Cell Contents:
usuario_id	genero					
1	O					
2	F					
3	O					
4	M					
5	M					